TECHNICAL HANDOVER DOCUMENT

Tata Trusts — Small Animal Hospital Mumbai

Website, Admin Panel & WhatsApp Chatbot Platform

Laravel 12 · PHP 8.2+ · MySQL

Prepared for project handover · Confidential

Contents

1. Project Overview
2. Technology Stack
3. System Architecture
4. Environments & Deployment
5. Configuration (`.env`)
6. Frontend (Public Website)
7. Backend (Admin Panel) & Modules
8. Appointment System
9. WhatsApp Chatbot
10. Notifications Module
11. Third-Party Integrations
12. Roles & Permissions (RBAC)
13. Database Overview
14. Scheduled Tasks & Queue
15. Logging & Auditing
16. Operational / Maintenance Notes
17. Known Constraints & Pending Items

1. Project Overview

A full web platform for **Small Animal Hospital Mumbai** (Tata Trusts), comprising a public marketing/informational website, a content-managed admin panel, an OTP-based online appointment booking system, and a reactive **WhatsApp chatbot** for pet parents.

Public website	Hospital info, specialities, team, facilities, events, blog, media, careers, contact & online appointment booking.
Admin panel	Role-based CMS to manage all website content, appointments, enquiries, notifications & reports.
Appointments	Mobile-OTP login → booking form → status workflow with SMS triggers.
WhatsApp bot	Menu-driven assistant: services, timings, emergency, FAQs, guided appointment booking, "talk to our team".
Live/reference site	sahmumbai.com (existing production reference). This build deploys to a UAT environment.

2. Technology Stack

Layer	Technology
Framework	Laravel 12 (PHP `^8.2`; UAT runs PHP 8.4)
Language	PHP 8.2+
Database	MySQL / MariaDB (charset `utf8mb4`, collation `utf8mb4_unicode_ci`)
Templating	Blade
Admin theme	"Riho" Bootstrap 5 admin template (jQuery, Feather icons, DataTables, Summernote/CKEditor, bootstrap-notify)
Frontend theme	Custom responsive theme (Bootstrap, jQuery, AOS, Slick)
Auth	Laravel session auth (admin) + custom mobile-OTP (appointment users)
HTTP client	Laravel HTTP (Guzzle) — used for SMS & WhatsApp APIs
Dev tooling	Tinker, Pint, Pail, Sail (dev only)

Scale: ~57 Eloquent models, ~87 migrations, ~40 backend controllers.

3. System Architecture

A conventional monolithic Laravel MVC application serving two surfaces from one codebase:

- **Frontend** — public routes (`routes/web.php`), `App\Http\Controllers\Frontend\*`, Blade views under `resources/views/frontend` + shared `components/frontend`.

- **Backend** — admin routes behind `auth` middleware, `App\Http\Controllers\Backend\*`, views under `resources/views/backend` + `components/backend`. Every module is gated by a `permission:` middleware alias.
- **Webhook** — `WhatsAppWebhookController` at `/webhooks/whatsapp` (public, CSRF-exempt) for inbound WhatsApp messages.

Key cross-cutting pieces

- `App\Support\WhatsAppBot` — the chatbot conversation engine.
- `App\Services\WhatsAppFortius` / `App\Services\MessageIndiaSms` — outbound API clients.
- `App\Support\CommunicationLogger` / `ActivityLogger` — central logging helpers.
- `App\Support\NotificationHooks` — raises in-app admin notifications on model events.
- `App\Http\Middleware\CheckPermission` — RBAC enforcement (alias `permission`).

4. Environments & Deployment

Environment	Details
Local (dev)	XAMPP, `http://127.0.0.1:8000` (artisan serve) or `localhost`. PHP 8.2.
UAT	Hosted at `anvayafoundation.com/tata_trusts_hospital`. PHP 8.4. Deployed via Git pull.
Repository	GitHub — branch `feature/whatsapp-chatbot` (active), `main`.

Deploy procedure (UAT)

1. `git pull origin feature/whatsapp-chatbot` (code + committed images).
2. `php artisan migrate` (schema changes).
3. `php artisan db:seed --class=ModulePermissionsSeeder` (permissions & role sync).
4. `php artisan config:clear && php artisan view:clear`.
5. For a queue worker (delayed WhatsApp menu): run `php artisan queue:work` or a cron/supervisor equivalent.

> **Important — data vs code.** Website *content* (doctor bios, page text, team members, speciality details) and *permission assignments* live in the **database**, not in Git. These do not transfer via `git pull` — they must be entered in the admin panel or applied via SQL on each environment. Uploaded images live in `public/` and are committed to Git.

5. Configuration (`.env`)

All credentials are sourced from `.env` (no hardcoded fallbacks in config). Keys are documented in `.env.example`.
Actual secret values are intentionally omitted from this document.

Group	Keys	Purpose
App	`APP_ENV`, `APP_DEBUG`, `APP_URL`, `APP_KEY`	Set `APP_ENV=production`, `APP_DEBUG=false`, correct `APP_URL` on live.
Database	`DB_*`	MySQL connection.
Mail (SMTP)	`MAIL_HOST/PORT/USERNAME/PASSWORD/ENCRYPTION/FROM_*`	Transactional email (SMTP).
Admin recipients	`MAIL_ADMIN_NOTIFICATION / _CONTACT / _CAREER / _APPOINTMENT`	Where each website form notification is sent. Set to the hospital's real inboxes in production.
SMS (MessageIndia)	`MESSAGEINDIA_USERNAME / _API_KEY / _SENDER_NAME / _PE_ID` + template IDs	OTP + appointment status SMS.
WhatsApp (Fortius)	`WHATSAPP_BASE_URL / _API_VERSION / _PHONE_NUMBER_ID / _TOKEN / _VERIFY_TOKEN`, optional `_BOOKING_URL / _WELCOME_IMAGE`	WhatsApp Cloud API via Fortius proxy.

> Because config has no fallbacks, a missing key makes that feature silently return `null`. Every environment's `.env` must set all required keys. Run `php artisan config:clear` after edits.

6. Frontend (Public Website)

Page	Route name
Home	`frontend.index`
Specialities (list + detail)	`frontend.specialities`, `frontend.specialities_details`
Our Facilities	`frontend.our_facilities`
Our Team	`frontend.our_team`
About Us	`frontend.about_us`
Events	`frontend.events`
Blog (list + detail)	`frontend.blogs`, `frontend.blog_details`
Media & Gallery	`frontend.media`, `frontend.gallery`
FAQs	`frontend.faqs`
Join Us (Careers)	`frontend.join_us`
Contact Us	`frontend.contact_us`
Appointment login / booking	`frontend.user_login` → `frontend.book_an_appointment`

Public forms (Contact, Careers/Job Application, Appointment) persist to the database, email the configured admin recipient, and send a user acknowledgement where applicable. A floating "Chat with us" WhatsApp button links to the hospital's WhatsApp number.

7. Backend (Admin Panel) & Modules

Login at `/login`. The sidebar is permission-driven — each admin sees only the modules their role allows.

Group	Modules
Dashboard	Overview tiles, today's appointments, status summary.
Notifications	Live activity bell + full notifications page (see §10).
Home	Banners, Short Introduction, Services, Facilities, Team, Testimonials, Board, Follow Us (8 home sections).
Specialities	Listing + Details (section content, services list, attached doctors; supports a "preventive care" layout variant).
Masters	Our Team, Testimonials, Our Facilities.
Content	About Us, Gallery, Events, Media, FAQs, Flyer, Contact Details, Privacy Policy.
Join Us	Join page content + Job Postings.
Blog	Categories, Listings, Details, Comments (moderation).
Appointments	Appointment Users, Appointments, WhatsApp Appointments, Manage Statuses (see §8).
Form Enquiries	Contact Enquiries, Job Applications, WhatsApp Queries.
Reports	Appointments & communication reports with CSV export.
Logs	Activity Log, Communication Logs (Super Admin only).
User	Roles, Users, Permissions (RBAC admin — see §12).

Admin list filters follow a project convention: AJAX POST, auto-apply on change, no full-page reload, with a results partial + loader. Most content modules follow a standard CRUD pattern (list / create / edit / delete) with image upload and rich-text where relevant.

7.1 Dashboard

- At-a-glance tiles: **Today's Appointments**, **Next 2 Days**, **Pending Appointments**, **Total Clients**.
- Today's appointment list and an appointment **status overview** summary.
- "View All Appointments" shortcut; live notification bell in the header.

7.2 Home Page Management

Controls every section of the public home page. Each is independently editable:

- **Banner Details** — hero slides/banner content (image, heading, text, call-to-action).
- **Short Introduction** — the intro block below the banner.
- **Services** — featured specialities/services shown on the home page.
- **Facilities** — featured facilities block.
- **Our Team** — selected doctors featured on home (toggle which appear).
- **Testimonials** — client testimonials carousel.
- **Our Board** — board/leadership block.
- **Follow Us** — social / follow-us section.

7.3 Specialities

- **Listing** — create/edit each speciality (name, slug, image). Drives the Specialities menu & cards.
- **Details** — the full speciality page: banner image, section image, section heading & description, a **services list**, and **attached doctors** (picked from Our Team, with an optional per-speciality bio override). Services render in balanced columns.
- Supports a **"Preventive Care" layout variant** — a speciality flagged as preventive reveals dedicated preventive sections, plan categories and disclaimer fields instead of the standard layout.

7.4 Masters (reusable content)

- **Our Team** — master directory of doctors/staff: name, designation (rich text), bio, photo, "show on team page" and "show on home" toggles. *Reused across the Team page and every speciality page* a doctor is attached to.
- **Testimonials** — master testimonials pool.
- **Our Facilities** — master facilities pool.

7.5 Content Pages

- **About Us** — about page content blocks.
- **Gallery** — galleries with multiple images; "show on home" toggle.
- **Events** — events CRUD; "show on home" toggle.
- **Media** — media / press items.
- **FAQs** — question & answer entries for the FAQ page.
- **Flyer** — promotional flyer / pop-up management.
- **Contact Details** — address, phone, emergency number, map URL, email(s), social links & the top ribbon items used across the site header/footer.
- **Privacy Policy** — policy documents/pages.

7.6 Join Us (Careers)

- **Page Details** — careers landing content.
- **Job Postings** — job roles CRUD (shown publicly on Join Us).
- Submitted applications appear under Form Enquiries → Job Applications (read-only, with résumé).

7.7 Blog

- **Category**, **Listing**, **Details** — full blog CRUD with categories, tags & rich content.
- **Comments** — moderation: approve/unapprove (toggle active) and delete user comments.

7.8 Form Enquiries (read-only inboxes)

- **Contact Enquiries** — website Contact-Us submissions (name, email, phone, subject, message) with detail view + reply shortcuts.
- **Job Applications** — career submissions with applicant details & résumé download.
- **WhatsApp Queries** — "Talk to our team" messages captured by the chatbot, with a "reply on WhatsApp" shortcut.

7.9 Reports

- **Appointments report** and **Communication report** with AJAX filters and **CSV export** of the filtered set.

7.10 Users, Roles & Permissions

- **Users** — create/manage admin accounts and assign a role.
- **Roles** — create roles and assign permissions via a matrix.
- **Permissions** — the permission catalogue (see §12).

Appointments, WhatsApp Appointments, Notifications, Activity Log & Communication Logs are detailed in §8–§10 and §15.

8. Appointment System

Booking flow (website)

1. User enters mobile on `/user-login` → OTP sent via SMS (MessageIndia).
2. OTP verified → an `AppointmentUser` (keyed by mobile) is found/created; session established.
3. Booking form submitted → an `AppointmentEnquiry` is created with a default status; confirmation SMS + emails fire.

Admin management

- **Appointments** — filterable list, per-appointment status change (recorded in an immutable `AppointmentStatusHistory`), CSV export.
- **Status triggers** — statuses can be flagged to fire transactional SMS (e.g. reschedule, cancellation). "Reschedule" statuses require a new future date.
- **Appointment Users** — client directory with appointment counts and a **Medium** indicator/filter (Website vs WhatsApp).
- **Manage Statuses** — master list of statuses (name, colour, default/active flags, SMS trigger, requires-date flag).

Website vs WhatsApp appointments

Each appointment carries a `source` (`website` / `whatsapp`) and a per-source reference number (`A-0001` for website, `WA-0001` for WhatsApp). The same controller/views power two scoped tabs — **Appointments** (website) and **WhatsApp Appointments** — with identical functionality. A WhatsApp booking becomes a real appointment when the user completes the pre-filled web form; extra chat-intake fields are folded into the appointment's reason.

9. WhatsApp Chatbot

Provider: Fortius (`waba.fortius.in.net`) — a transparent proxy over Meta's WhatsApp Cloud API, so payloads follow Meta's format exactly. The chatbot runs *inside the 24-hour customer-service window* (user messages first), so no approved templates are needed for replies.

Components

- `WhatsAppWebhookController` — inbound webhook at `/webhooks/whatsapp` (GET = verify handshake, POST = messages). Requires a public HTTPS URL (UAT/production only).
- `App\Support\WhatsAppBot` — conversation engine: greeting + tappable list menu, services (live from DB), timings/emergency (from Contact Details), FAQs, blog, booking, "talk to our team". Per-user state in `whatsapp_conversations`.
- `App\Services\WhatsAppFortius` — sends text / image / interactive buttons / lists; normalises numbers; logs every send.
- `php artisan whatsapp:simulate` — local tester that drives the bot and prints replies (no credentials needed).

Guided booking flow

1. New / Existing client branch. Existing clients are verified against `AppointmentUser` by number; an already-registered number is prevented from registering again.
2. Step-by-step intake (parent & pet details), reason, preferred day/time.

3. **Review step** — a summary with a Confirm button; the user can type a field name (e.g. "email") to correct a single answer.
4. On confirm → a `WhatsAppBookingRequest` is saved + Customer Care emailed, and the user receives a confirmation + a login link to complete the booking online (the booking becomes a real appointment on web-form submit). Re-using a completed link shows an "already booked — start fresh" notice.
5. The main menu is re-sent ~30s later via a queued job (`SendWhatsAppMainMenu`).

"Talk to our team"

Captured messages are saved to `whatsapp_team_queries`, emailed to Customer Care, and listed under Form Enquiries → WhatsApp Queries.

> **Delivery constraints:** the sending number is a Meta *test* number — it only delivers to *allow-listed* recipients, and free-form replies require an open 24-hour session (the user must message first). A verified business number removes these limits. Business-initiated messages (e.g. reminders) require approved Meta templates (planned, not built).

10. Notifications Module

A shared-inbox admin notification system that alerts staff to new activity in real time.

- **Sources** — model `created` events (via `NotificationHooks`) on Contact Enquiries, Appointments, Job Applications and WhatsApp Queries raise an `AdminNotification`.
- **Live bell** — header bell with unread badge; AJAX **polling every ~20s** (no WebSockets — a deliberate choice for shared hosting), so new items appear without reloading.
- **Toasts + chime** — a clean toast and a soft sound on new activity; a periodic in-app reminder re-surfaces unread items.
- **Notifications page** — full list with summary tiles, filters, mark-all-read.

Available to every admin (not permission-gated, like the Dashboard). Reminders are in-panel only (no email reminders).

11. Third-Party Integrations

Integration	Used for	Notes
MessageIndia SMS	Login OTPs; appointment confirmation / reschedule / cancellation SMS	DLT-registered templates (PE ID + template IDs in `.env`).
SMTP email	Form notifications to admin + user acknowledgements	Branded HTML mailables.
WhatsApp (Fortius / Meta Cloud API)	Chatbot send + inbound webhook	See §9.

Every outbound SMS, email and WhatsApp message is recorded in `communication_logs` (channel, type, recipient, status, provider response, error), viewable under Communication Logs and Reports.

12. Roles & Permissions (RBAC)

Permissions are defined per module as `{slug}.{view|create|edit|delete}` in `ModulePermissionsSeeder`, enforced in three coordinated places: the seeder catalogue, route `permission:` middleware, and sidebar `$can()` gates.

Role	Access
Super Admin	All permissions (also short-circuits to allow everything). Only role with Activity Log, Communication Logs, and delete actions.
Admin	View / create / edit on every module (no delete).
Front Office	Appointments, Appointment Users, Statuses, Contact Enquiries, Reports, Dashboard + WhatsApp Appointments (view/edit) & WhatsApp Queries (view).
Career Admin	Job Applications, Job Roles, Join Page.
User (standard)	Dashboard only.

> **When adding a module:** add it to `ModulePermissionsSeeder` (`$modules`), gate its routes with `permission:{slug}.{action}` , add a sidebar `$can()` gate, then re-run the seeder. Custom roles (Front Office, Career Admin) are *not* auto-synced by the seeder — grant new permissions to them manually.

Permissions table + role assignments are database rows (`permissions` , `roles` , `permission_role`). Run the seeder on each environment; grant custom-role permissions via admin UI or SQL.

13. Database Overview

~87 migrations. Key tables by area:

Area	Tables
Access control	`users`, `roles`, `permissions`, `permission_role` (+ role↔user)
Appointments	`appointment_users`, `appointment_enquiries`, `appointment_statuses`, `appointment_status_histories`, `appointment_otps`
WhatsApp	`whatsapp_conversations`, `whatsapp_booking_requests`, `whatsapp_team_queries`
Notifications & logs	`admin_notifications`, `communication_logs`, `activity_logs`
Enquiries	`contact_enquiries`, `job_applications`
Content / CMS	`specialities`, `speciality_details`, `our_teams`, `home_*`, `blog_*`, `events`, `galleries`, `media`, `faqs`, `about_us`, `flyers`, `contact_details`, `privacy_policies`, `join_page*`, `testimonials`, `master_facilities` …

Most content tables use a soft-delete convention via a `deleted_by` column (queries filter `whereNull('deleted_by')`); some models also use Laravel `SoftDeletes`.

14. Scheduled Tasks & Queue

- **Queue** — the delayed WhatsApp main-menu uses a queued job (`SendWhatsAppMainMenu`, 30s delay). With the `sync` driver it sends immediately; a real delay needs `QUEUE_CONNECTION=database` (or redis) + a running worker (`php artisan queue:work` via supervisor/cron).
- **Scheduler** — defined in `routes/console.php`. If scheduled tasks are used, add a server cron: `* * * * * php artisan schedule:run`.

15. Logging & Auditing

- **Activity Log** — audit trail of admin create/update/delete across tracked models, plus login / logout / failed-login events (Super Admin view).
- **Communication Logs** — every SMS / email / WhatsApp send attempt with status, error and provider response (Super Admin view).
- **Application logs** — standard Laravel logs under `storage/logs`.

16. Operational / Maintenance Notes

- **Production checklist:** `APP_ENV=production`, `APP_DEBUG=false`, correct `APP_URL` (affects OTP/booking links & assets), real `MAIL_ADMIN_*` inboxes, verified WhatsApp business number.

- **After** `.env` **or route changes:** `php artisan config:clear` (and `route:clear` / `view:clear` as needed).
- **WhatsApp webhook** must be registered in the Fortius panel to the deployed HTTPS URL with the matching verify token, and "Wrapper Client" enabled so inbound is forwarded to the app.
- **Content & permissions are data** — manage via the admin panel (or SQL), per environment; they are not carried by Git.
- **Image assets** are stored under `public/` (e.g. `public/our-team`, `public/home/*`) and are committed to Git.

17. Known Constraints & Pending Items

- **WhatsApp test number** — deliver only to allow-listed recipients; move to a verified business number for general availability.
- **Outbound WhatsApp templates** — appointment status notifications over WhatsApp (confirmed / rescheduled / cancelled) require approved Meta templates — planned, not yet built (SMS equivalents exist).
- **Queue worker** — required for the true 30-second delayed menu; otherwise it sends immediately.
- **Content parity** — doctor bios / speciality text / images are synced per environment as data; keep in mind when promoting UAT → production.

End of document · Confidential — Small Animal Hospital Mumbai technical handover.